Análisis de la cesta de la compra (*Market Basket Analysis*) — Sano y Fresco

Versión corregida y explicada paso a paso.

Objetivo

Descubrir **qué productos se compran juntos** en los pedidos de Sano y Fresco, para:

- colocar productos relacionados cerca en la tienda o en la web,
- hacer recomendaciones del tipo *"quien compra A, suele comprar B"*,
- diseñar promociones de productos que se compran juntos.

Las tres métricas clave (con un ejemplo)

Imagina 100 pedidos. En 20 hay **cilantro** (A), en 10 hay **limones** (B) y en 5 hay **los dos**.

Métrica	Qué responde	Fórmula	Ejemplo
Soporte	¿Con qué frecuencia aparece?	pedidos con X / total pedidos	Soporte(A) = 20/100 = 20 %. Soporte(A y B) = 5/100 = 5 %
Confianza A → B	De los que compran A, ¿cuántos compran también B?	Soporte(A y B) / Soporte(A)	5 / 20 = 25 %
Lift	¿Comprar A hace **más probable** comprar B que si fuera al azar?	Confianza(A → B) / Soporte(B)	25 % / 10 % = **2,5**

Cómo leer el lift:

- **Lift > 1**: los productos se compran juntos **más** de lo esperado por azar → asociación real. Útil para recomendar.
- **Lift = 1**: no hay relación; se compran juntos lo que tocaría por casualidad.
- **Lift < 1**: se compran juntos **menos** de lo esperado → uno parece **sustituir** al otro (por ejemplo, dos tipos de plátano).

⚠️ La confianza **no es simétrica**: Confianza(A → B) ≠ Confianza(B → A). El lift **sí** es simétrico.

Índice

1. Configuración e importación de librerías
2. Carga de los datos desde la base de datos
3. Revisión y calidad de los datos
4. Construcción de la matriz de cestas (pedido × producto)
5. Soporte de cada producto
6. Reglas de asociación: soporte, confianza y lift
7. Comprobación: los resultados coinciden con las funciones originales
8. Filtrado de las reglas útiles
9. Visualización
10. Enriquecer las reglas con los datos de producto
11. Exportación a CSV
12. Cómo interpretar los resultados

> **Errores corregidos respecto a la versión anterior** (marcados con 🔧 a lo largo del notebook):
>
> - Con `combinations` **solo se calculaba un sentido** de cada regla (A → B, pero nunca B → A). Como la confianza no es simétrica, **faltaba la mitad de las reglas**.
> - La lista final mezclaba reglas con **lift < 1** (productos que *no* se compran juntos) con las reglas útiles.
> - Faltaba el **soporte de la regla** (cuántos pedidos tienen los dos productos), imprescindible para saber si una regla es fiable.
> - El **lift se redondeaba a 1 decimal**, lo que no deja distinguir, por ejemplo, 0,96 de 1,04.
> - Solo se enriquecía el **antecedente**; el consecuente quedaba sin sus identificadores.
> - El CSV se exportaba **sin** `utf-8-sig`, así que las tildes ("Orgánica") salen mal al abrirlo en Excel.
> - Unir los productos con comas y separarlos después **se rompe** si algún nombre contiene una coma.
> - El cálculo era muy lento (filtraba los 2 millones de pedidos varias veces por cada pareja de productos). Ahora se hace con una sola multiplicación de matrices.

Resultados con los datos reales

Ejecutado con `sanoyfresco.db`: 4.975.718 líneas de venta, 2.060.188 pedidos, 181.903 clientes y 40 productos (año 2023).

	Versión original	Versión corregida
Reglas calculadas	780 (un solo sentido por pareja)	1.560 (los dos sentidos)
Reglas en la tabla final	399 (solo filtro de confianza > 5 %)	**258 reglas útiles** (soporte ≥ 0,5 %, confianza ≥ 5 % y lift > 1)
Reglas con lift < 1 mezcladas con las útiles	48	0 (las 100 parejas con lift < 1 van en una tabla aparte)
Confianza media	8,33 %	13,5 %
Lift medio	1,46	1,51

Las asociaciones más fuertes son **Cilantro ↔ Limones** (lift 3,58), **Ajo ↔ Cebolla amarilla** (3,53) y **Apio ↔ Cebolla amarilla** (2,84). Las cifras de la versión original (399 reglas, confianza media 8,33 % y lift medio 1,46) se han reproducido aplicando su método a estos mismos datos.

1. Configuración e importación de librerías

🔧 Todas las librerías se importan aquí, una sola vez. Antes `combinations` se importaba en mitad del notebook.

Si trabajas en Google Colab

1. Sube `sanoyfresco.db` al panel de **Archivos** (icono de la carpeta, a la izquierda).
2. Si al cargarla da un error de que la base de datos **está corrupta**, asegúrate de que ha terminado de subirse; después haz clic en los 3 puntitos del nombre de la base de datos y en **Actualizar**.

```
import sqlite3
import pathlib
from contextlib import closing

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# --- Rutas ---
RUTA_BD = pathlib.Path('sanoyfresco.db')
RUTA_CSV_REGLAS = pathlib.Path('reglas_market_basket.csv')   # el archivo que carga Power BI
RUTA_SECCIONES = pathlib.Path('secciones.xlsx')   # opcional: nombre e imagen de cada sección

# --- Umbrales para filtrar reglas (se explican en el paso 8) ---
SOPORTE_REGLA_MIN = 0.005   # la pareja aparece al menos en el 0,5 % de los pedidos
CONFIANZA_MIN = 0.05        # al menos el 5 % de los que compran A compran también B
LIFT_MIN = 1.0              # solo asociaciones positivas (más de lo esperado por azar)
```

2. Carga de los datos desde la base de datos

🔧 Tres mejoras sobre el original:

1. **Comprobar que el archivo existe antes de conectar.** `sqlite3.connect()` tiene una trampa: si el archivo no existe, **crea una base de datos vacía** sin avisar. El error aparecería después, en la consulta, con un mensaje confuso (`no such table: tickets`). Así el error es claro desde el principio.
2. `with closing(...)` cierra la conexión automáticamente al terminar, **aunque la consulta falle**. Antes, si la consulta daba error, la conexión se quedaba abierta.
3. **Seleccionar solo las columnas que se usan** en vez de `SELECT *` . La tabla tiene casi 5 millones de filas: cada columna de menos ahorra memoria (en Colab la memoria es limitada).

```
if not RUTA_BD.exists():
    raise FileNotFoundError(
        f'No se encuentra {RUTA_BD}. Súbelo a la misma carpeta que el notebook '
        '(en Colab: panel de Archivos, a la izquierda).')

consulta = '''
    SELECT id_pedido, id_cliente, fecha, hora,
           id_departamento, id_seccion, id_producto, nombre_producto,
           precio_unitario, cantidad, precio_total
    FROM tickets
'''

with closing(sqlite3.connect(RUTA_BD)) as conexion:
    df = pd.read_sql_query(consulta, conexion)

print(f'Filas: {len(df):,} | Columnas: {df.shape[1]}')
df.head()
```

```
Filas: 4,975,718 | Columnas: 11
```

	id_pedido	id_cliente	fecha	hora	id_departamento	id_seccion	id_producto	nombre_producto	prec
0	1	112108	2023-02-02 00:00:00	10	4	83	49683	Pepino Kirby	
1	1	112108	2023-02-02 00:00:00	10	4	24	13176	Bolsa de Bananas Orgánicas	
2	1	112108	2023-02-02 00:00:00	10	4	24	47209	Aguacate Hass Orgánico	
3	2	202279	2023-02-03 00:00:00	9	4	83	28985	Col Rizada Orgánica de Michigan	
4	2	202279	2023-02-03 00:00:00	9	4	83	17794	Zanahorias	

En este análisis solo usamos `id_pedido` y `nombre_producto` (más los identificadores de producto para enriquecer las reglas al final). Se cargan también las demás columnas para poder **revisar la calidad de los datos** en el paso 3. Si te quedas sin memoria, puedes quitar de la consulta `id_cliente` , `fecha` , `hora` y las columnas de precio; en ese caso, salta también las comprobaciones del paso 3 que las usan (fecha y coherencia de precios).

3. Revisión y calidad de los datos

🔧 Este paso no existía. Antes de calcular nada hay que comprobar que los datos son coherentes:

- tipos de datos y valores nulos,
- que el precio total = precio unitario × cantidad,
- si un mismo producto aparece **varias veces en el mismo pedido** (para la cesta solo importa si está o no, no cuántas veces),
- que cada nombre de producto corresponde a **un solo** `id_producto` (lo necesitaremos en el paso 10).

La conversión de `fecha` a tipo fecha se mantiene, aunque este análisis no la usa: es útil si quieres repetir el análisis por meses o temporadas.

```
df['fecha'] = pd.to_datetime(df['fecha'])
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4975718 entries, 0 to 4975717
Data columns (total 11 columns):
 #   Column           Dtype         
---  ------           -----         
 0   id_pedido        int64         
 1   id_cliente       int64         
 2   fecha            datetime64[us]
 3   hora             int64         
 4   id_departamento  int64         
 5   id_seccion       int64         
 6   id_producto      int64         
 7   nombre_producto  str           
 8   precio_unitario  float64       
 9   cantidad         int64         
 10  precio_total     float64       
dtypes: datetime64[us](1), float64(2), int64(7), str(1)
memory usage: 417.6 MB
```

```
print('Valores nulos por columna:')
print(df.isna().sum())

# Precio total = precio unitario × cantidad (con tolerancia por redondeo)
coherente = np.isclose(df['precio_total'], df['precio_unitario'] * df['cantidad'], atol=0.01)
print(f'\nFilas con precio_total incoherente: {(~coherente).sum():,}')

# Mismo producto repetido dentro de un mismo pedido
repetidos = df.duplicated(subset=['id_pedido', 'nombre_producto']).sum()
print(f'Líneas con un producto repetido en el mismo pedido: {repetidos:,}')

# ¿Cada nombre de producto tiene un único id_producto?
ids_por_nombre = df.groupby('nombre_producto')['id_producto'].nunique()
print(f'Nombres de producto con más de un id_producto: {(ids_por_nombre > 1).sum()}')

print(f'\nPedidos distintos:   {df["id_pedido"].nunique():,}')
print(f'Productos distintos: {df["nombre_producto"].nunique()}')
print(f'Productos por pedido (media): {df.groupby("id_pedido")["nombre_producto"].nunique().m
```

```
Valores nulos por columna:
```

```
id_pedido          0
id_cliente         0
fecha              0
hora               0
id_departamento    0
id_seccion         0
id_producto        0
nombre_producto    0
precio_unitario    0
cantidad           0
precio_total       0
dtype: int64

Filas con precio_total incoherente: 0
Líneas con un producto repetido en el mismo pedido: 0
Nombres de producto con más de un id_producto: 0
Pedidos distintos:   2,060,188
Productos distintos: 40
Productos por pedido (media): 2.42
```

```python
# Distribución del número de productos distintos por pedido
productos_por_pedido = df.groupby('id_pedido')['nombre_producto'].nunique()
print(productos_por_pedido.value_counts().sort_index().rename('pedidos').to_string())
print(f'\nPedidos con un solo producto: {(productos_por_pedido == 1).mean():.1%}')
```

```
nombre_producto
1     806354
2     509428
3     315429
4     190879
5     110994
6      62172
7      33215
8      16760
9       8268
10      3850
11      1637
12       717
13       301
14       118
15        41
16        19
18         3
19         3

Pedidos con un solo producto: 39.1%
```

Los pedidos con **un solo producto** no pueden contribuir a ninguna pareja, pero se mantienen en el total de pedidos: el soporte se calcula sobre **todos** los pedidos, igual que en la versión original.

4. Construcción de la matriz de cestas (pedido × producto)

Para calcular las reglas necesitamos una tabla con:

- **una fila por pedido**,
- **una columna por producto**,
- `True` si el producto está en el pedido y `False` si no.

🔧 Por qué no unir con comas y separar después

En el original se hacía:

```
df_cesta.groupby('id_pedido')['nombre_producto'].apply(lambda producto:
','.join(producto))
df_agrupado.str.get_dummies(sep=',')
```

Funciona con estos 40 productos, pero tiene dos problemas:

1. Si algún nombre de producto **contuviera una coma** (por ejemplo, "Queso, curado"), se partiría en dos productos falsos ("Queso" y "curado").
2. Es lento y gasta mucha memoria: crea un texto largo por cada uno de los 2 millones de pedidos y luego lo vuelve a trocear. Además, `get_dummies` genera números enteros de 8 bytes donde basta un `True` / `False` de 1 byte.

Cómo se hace ahora

1. `drop_duplicates()`: cada producto cuenta **una sola vez por pedido**.
2. `pd.factorize()` asigna a cada pedido y a cada producto un número de posición (0, 1, 2…).
3. Se crea una matriz de `False` y se pone `True` en cada posición (pedido, producto) que existe.

El resultado es la misma tabla que antes, pero sin depender de las comas y ocupando 8 veces menos memoria.

```
lineas = df[['id_pedido', 'nombre_producto']].drop_duplicates()

posicion_pedido, pedidos = pd.factorize(lineas['id_pedido'], sort=True)
posicion_producto, productos = pd.factorize(lineas['nombre_producto'], sort=True)

matriz = np.zeros((len(pedidos), len(productos)), dtype=bool)
matriz[posicion_pedido, posicion_producto] = True

cestas = pd.DataFrame(matriz, index=pd.Index(pedidos, name='id_pedido'), columns=productos)

print(f'Matriz de cestas: {cestas.shape[0]:,} pedidos × {cestas.shape[1]} productos')
print(f'Memoria: {cestas.memory_usage(deep=True).sum() / 1e6:,.1f} MB')
cestas.head()
```

```
Matriz de cestas: 2,060,188 pedidos × 40 productos
Memoria: 98.9 MB
```

id_pedido	Agua con Gas de Pomelo	Aguacate Hass Orgánico	Aguacate Orgánico	Ajo Orgánico	Apio Orgánico en Ramillete Pequeño	Arándanos Orgánicos	Banana	Bolsa de Bananas Orgánicas	Calabacín Orgánico
1	False	True	False	False	False	False	False	True	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
5	False	True	False	False	False	False	False	True	False
10	False	False	True	False	False	False	True	False	False

5 rows × 40 columns

```
# Comprobación: cada pedido tiene tantos True como productos distintos
assert (cestas.sum(axis=1).values == productos_por_pedido.loc[cestas.index].values).all()
print('✅ La matriz de cestas coincide con los productos de cada pedido')
```

```
✅ La matriz de cestas coincide con los productos de cada pedido
```

5. Soporte de cada producto

El **soporte** de un producto es la proporción de pedidos en los que aparece. Como la matriz es de `True` / `False` (y Python trata `True` como 1), la **media** de cada columna es directamente esa proporción.

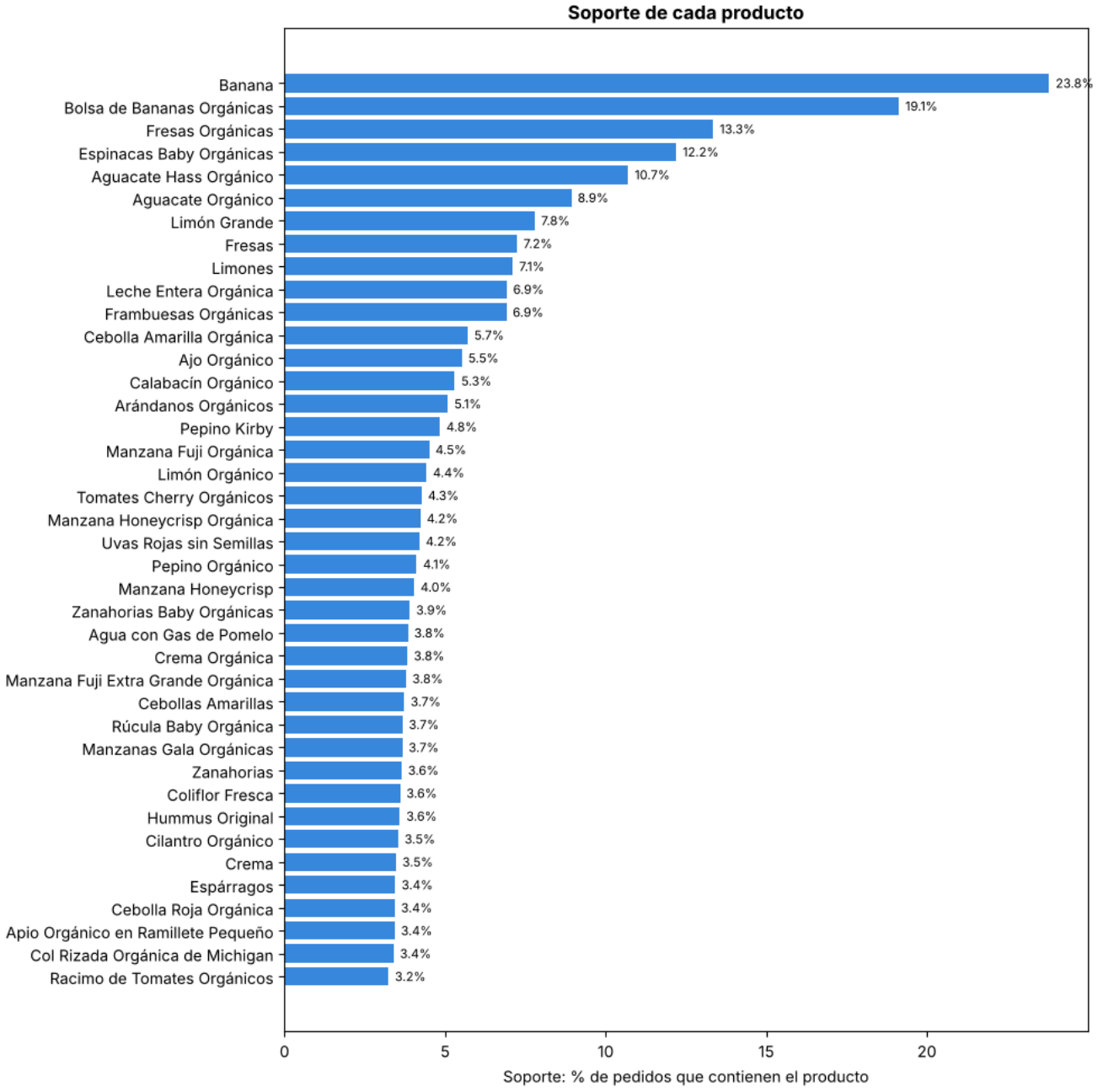
Es la base de todo lo demás: un producto que está en casi todos los pedidos (como el plátano) aparecerá junto a cualquier otro "por casualidad".

```
soporte = cestas.mean()
(soporte.sort_values(ascending=False) * 100).round(2).rename('soporte_%').to_frame()
```

	soporte_%
Banana	23.81
Bolsa de Bananas Orgánicas	19.13
Fresas Orgánicas	13.35
Espinacas Baby Orgánicas	12.20
Aguacate Hass Orgánico	10.70
Aguacate Orgánico	8.93
Limón Grande	7.79
Fresas	7.24
Limones	7.11
Leche Entera Orgánica	6.92
Frambuesas Orgánicas	6.91
Cebolla Amarilla Orgánica	5.71
Ajo Orgánico	5.52
Calabacín Orgánico	5.30
Arándanos Orgánicos	5.09
Pepino Kirby	4.84
Manzana Fuji Orgánica	4.50
Limón Orgánico	4.42
Tomates Cherry Orgánicos	4.27
Manzana Honeycrisp Orgánica	4.23
Uvas Rojas sin Semillas	4.20
Pepino Orgánico	4.11
Manzana Honeycrisp	4.04
Zanahorias Baby Orgánicas	3.90
Agua con Gas de Pomelo	3.84
Crema Orgánica	3.83
Manzana Fuji Extra Grande Orgánica	3.78
Cebollas Amarillas	3.73
Rúcula Baby Orgánica	3.67
Manzanas Gala Orgánicas	3.67
Zanahorias	3.65
Coliflor Fresca	3.60
Hummus Original	3.59
Cilantro Orgánico	3.54
Crema	3.47
Espárragos	3.45
Cebolla Roja Orgánica	3.43
Apio Orgánico en Ramillete Pequeño	3.43

	soporte_%
Col Rizada Orgánica de Michigan	3.40
Racimo de Tomates Orgánicos	3.24

```
fig, ax = plt.subplots(figsize=(10, 10))
s = (soporte.sort_values() * 100)
ax.barh(s.index, s.values, color='#378ADD')
ax.set_xlabel('Soporte: % de pedidos que contienen el producto')
ax.set_title('Soporte de cada producto', fontweight='bold')
for y, v in enumerate(s.values):
    ax.text(v + 0.2, y, f'{v:.1f}%', va='center', fontsize=8)
plt.tight_layout()
plt.show()
```



6. Reglas de asociación: soporte, confianza y lift

🔧 El error principal: `combinations` frente a `permutations`

El original recorría las parejas con `combinations(columnas, 2)`. `combinations` devuelve cada pareja **una sola vez y en un único orden**:

```
from itertools import combinations, permutations
list(combinations(['Ajo', 'Limones', 'Zanahorias'], 2))
# [('Ajo', 'Limones'), ('Ajo', 'Zanahorias'), ('Limones', 'Zanahorias')]   → 3 reglas
list(permutations(['Ajo', 'Limones', 'Zanahorias'], 2))
# [('Ajo', 'Limones'), ('Ajo', 'Zanahorias'), ('Limones', 'Ajo'),
#  ('Limones', 'Zanahorias'), ('Zanahorias', 'Ajo'), ('Zanahorias', 'Limones')]   → 6 reglas
```

Como las columnas están en **orden alfabético**, el antecedente siempre era el producto que va antes en el abecedario. Se calculaba "Ajo → Limones" pero **nunca "Limones → Ajo"**, y la confianza de las dos reglas es distinta. Con 40 productos hay 1.560 reglas posibles y solo se calculaban 780.

Comprobado con tu archivo `reglas.csv` original (el que generó la versión anterior):

- De las 399 reglas, **ninguna** tiene su regla inversa en la tabla.
- En **las 399**, el antecedente va alfabéticamente antes que el consecuente.
- **48** reglas tenían lift < 1 y **26** aparecían con lift exactamente "1.0" por el redondeo a un decimal (ver paso 8).

🔧 Cálculo rápido con matrices

El original filtraba la tabla de 2 millones de pedidos **varias veces por cada pareja** (en `confianza()`, en `lift()` y otra vez en el bucle). Con 40 productos son miles de filtros sobre millones de filas.

Hay una forma mucho más rápida: si `X` es la matriz de cestas (con 0 y 1), la multiplicación `X.T @ X` da en una sola operación una tabla 40 × 40 donde la casilla (A, B) es **el número de pedidos que contienen A y B a la vez**. La diagonal (A, A) es el número de pedidos con A.

A partir de esa tabla, todas las métricas son divisiones:

- soporte(A y B) = pedidos con A y B / total de pedidos
- confianza(A → B) = pedidos con A y B / pedidos con A
- lift(A → B) = confianza(A → B) / soporte(B)

```
# float32 en vez de enteros: numpy multiplica matrices decimales con rutinas muy optimizadas.
# Con 2 millones de pedidos tarda menos de 1 segundo (con enteros, más de 30).
# Es exacto: float32 representa sin error cualquier entero hasta 16.777.216,
# y ningún recuento puede superar el número de pedidos.
X = cestas.to_numpy(dtype=np.float32)
n_pedidos = X.shape[0]

conjuntos = (X.T @ X).round().astype(np.int64)   # (A, B) = nº de pedidos con A y B
pedidos_con = np.diag(conjuntos)                  # (A, A) = nº de pedidos con A

conjuntos_df = pd.DataFrame(conjuntos, index=productos, columns=productos)
print('Pedidos que contienen a la vez cada pareja (primeras 5 × 5):')
conjuntos_df.iloc[:5, :5]
```

```
Pedidos que contienen a la vez cada pareja (primeras 5 × 5):
```

	Agua con Gas de Pomelo	Aguacate Hass Orgánico	Aguacate Orgánico	Ajo Orgánico	Apio Orgánico en Ramillete Pequeño
Agua con Gas de Pomelo	79101	7544	6518	3761	1990
Aguacate Hass Orgánico	7544	220502	940	18316	12156
Aguacate Orgánico	6518	940	183961	14199	6267
Ajo Orgánico	3761	18316	14199	113759	9505
Apio Orgánico en Ramillete Pequeño	1990	12156	6267	9505	70621

```
# Pasar de la tabla 40 × 40 a una fila por regla A → B (todas las parejas ordenadas, sin A →
reglas = (conjuntos_df
          .rename_axis(index='antecedente', columns='consecuente')
          .stack()
          .rename('pedidos_ac')
          .reset_index())
reglas = reglas[reglas['antecedente'] != reglas['consecuente']].reset_index(drop=True)

reglas['pedidos_a'] = reglas['antecedente'].map(dict(zip(productos, pedidos_con)))
reglas['soporte_a'] = reglas['antecedente'].map(soporte)
reglas['soporte_c'] = reglas['consecuente'].map(soporte)
reglas['soporte_ac'] = reglas['pedidos_ac'] / n_pedidos
reglas['confianza'] = reglas['pedidos_ac'] / reglas['pedidos_a']
reglas['lift'] = reglas['confianza'] / reglas['soporte_c']

print(f'Reglas calculadas: {len(reglas):,} (= {len(productos)} × {len(productos) - 1})')
reglas.sort_values('lift', ascending=False).head(10)
```

```
Reglas calculadas: 1,560 (= 40 × 39)
```

	antecedente	consecuente	pedidos_ac	pedidos_a	soporte_a	soporte_c	soporte_ac	confianza	
478	Cilantro Orgánico	Cebolla Roja Orgánica	9046	72908	0.035389	0.034297	0.004391	0.124074	3.(
401	Cebolla Roja Orgánica	Cilantro Orgánico	9046	70658	0.034297	0.035389	0.004391	0.128025	3.(
948	Limones	Cilantro Orgánico	18527	146437	0.071079	0.035389	0.008993	0.126519	3.5
491	Cilantro Orgánico	Limones	18527	72908	0.035389	0.071079	0.008993	0.254115	3.5
354	Cebolla Amarilla Orgánica	Ajo Orgánico	22886	117561	0.057063	0.055218	0.011109	0.194673	3.5
125	Ajo Orgánico	Cebolla Amarilla Orgánica	22886	113759	0.055218	0.057063	0.011109	0.201180	3.5
193	Apio Orgánico en Ramillete Pequeño	Zanahorias	8487	70621	0.034279	0.036474	0.004120	0.120177	3.2
1486	Zanahorias	Apio Orgánico en Ramillete Pequeño	8487	75143	0.036474	0.034279	0.004120	0.112945	3.2
188	Apio Orgánico en Ramillete Pequeño	Pepino Orgánico	8967	70621	0.034279	0.041141	0.004353	0.126974	3.0
1291	Pepino Orgánico	Apio Orgánico en Ramillete Pequeño	8967	84758	0.041141	0.034279	0.004353	0.105795	3.0

Comprobamos que la confianza **no es simétrica** pero el lift **sí**, con la pareja de mayor lift:

```
top = reglas.sort_values('lift', ascending=False).iloc[0]
a, b = top['antecedente'], top['consecuente']
ida = reglas[(reglas['antecedente'] == a) & (reglas['consecuente'] == b)].iloc[0]
vuelta = reglas[(reglas['antecedente'] == b) & (reglas['consecuente'] == a)].iloc[0]

print(f'{a} → {b}: confianza {ida["confianza"]:.1%} | lift {ida["lift"]:.3f}')
print(f'{b} → {a}: confianza {vuelta["confianza"]:.1%} | lift {vuelta["lift"]:.3f}')
print('\nCon combinations solo se habría calculado una de estas dos reglas.')
```

```
Cilantro Orgánico → Cebolla Roja Orgánica: confianza 12.4% | lift 3.618
Cebolla Roja Orgánica → Cilantro Orgánico: confianza 12.8% | lift 3.618

Con combinations solo se habría calculado una de estas dos reglas.
```

7. Comprobación: los resultados coinciden con las funciones originales

Cuando cambias la forma de calcular algo, hay que **demostrar** que el resultado es el mismo. Aquí están las funciones `confianza()` y `lift()` del notebook original (sin cambios, salvo que reciben la tabla como parámetro) y se comparan con el cálculo nuevo en varias reglas al azar.

🔧 Sobre las funciones originales: eran **correctas**. El error no estaba en las fórmulas, sino en recorrer las parejas en un solo sentido.

```
def confianza(tabla, antecedente, consecuente):
    # Función original: probabilidad de que se compren los dos productos
    # dividido por la probabilidad de que se compre el producto antecedente
    conjunto_ac = tabla[(tabla[antecedente] == 1) & (tabla[consecuente] == 1)]
    return len(conjunto_ac) / tabla[antecedente].sum()


def lift(tabla, antecedente, consecuente):
    # Función original: cuánto aumenta la probabilidad de venta conjunta
    # en comparación con la venta individual aleatoria
    soporte_a = tabla[antecedente].mean()
    soporte_c = tabla[consecuente].mean()
    conteo_ac = len(tabla[(tabla[antecedente] == 1) & (tabla[consecuente] == 1)])
    soporte_ac = conteo_ac / len(tabla)
    return soporte_ac / (soporte_a * soporte_c)


muestra = reglas.sample(5, random_state=42)
for _, r in muestra.iterrows():
    conf_original = confianza(cestas, r['antecedente'], r['consecuente'])
    lift_original = lift(cestas, r['antecedente'], r['consecuente'])
    assert np.isclose(conf_original, r['confianza']) and np.isclose(lift_original, r['lift'])
    print(f"✅ {r['antecedente']} → {r['consecuente']}: "
          f"confianza {r['confianza']:.4f} | lift {r['lift']:.4f}")
print('\nEl cálculo con matrices da exactamente lo mismo que las funciones originales.')
```

```
✅ Zanahorias Baby Orgánicas → Apio Orgánico en Ramillete Pequeño: confianza 0.0515 | lift 1.5026
✅ Limón Orgánico → Cilantro Orgánico: confianza 0.0628 | lift 1.7735
✅ Cebolla Amarilla Orgánica → Ajo Orgánico: confianza 0.1947 | lift 3.5256
✅ Espinacas Baby Orgánicas → Banana: confianza 0.2120 | lift 0.8906
✅ Crema Orgánica → Fresas: confianza 0.0489 | lift 0.6747

El cálculo con matrices da exactamente lo mismo que las funciones originales.
```

8. Filtrado de las reglas útiles

🔧 Qué fallaba en el filtro original

El original solo exigía **confianza > 5 %** y luego ordenaba por lift. Eso dejaba pasar:

1. **Reglas con lift < 1.** En la tabla final había reglas como "Aguacate Hass Orgánico → Banana" con lift 0,6: los que compran aguacate compran plátano **menos** de lo normal. Eso no es una asociación para recomendar; es lo contrario. Aparecían porque el plátano está en 1 de cada 4 pedidos, así que casi cualquier producto tiene más de un 5 % de confianza hacia él.
2. **Reglas con muy pocos pedidos detrás.** Sin mirar el soporte de la pareja, una regla basada en un puñado de pedidos puede tener una confianza alta por casualidad.

Los tres umbrales (definidos en el paso 1)

- `SOPORTE_REGLA_MIN` (0,5 %): la pareja tiene que aparecer en un número mínimo de pedidos para que la regla sea **fiable**.
- `CONFIANZA_MIN` (5 %): el mismo umbral que el original.
- `LIFT_MIN` (1): solo asociaciones **positivas**.

Son decisiones de negocio: puedes cambiarlos arriba y volver a ejecutar.

```
reglas_utiles = reglas[
    (reglas['soporte_ac'] >= SOPORTE_REGLA_MIN) &
    (reglas['confianza'] >= CONFIANZA_MIN) &
    (reglas['lift'] > LIFT_MIN)
].sort_values(['lift', 'confianza'], ascending=False).reset_index(drop=True)

reglas_negativas = reglas[
    (reglas['soporte_ac'] >= SOPORTE_REGLA_MIN) & (reglas['lift'] < 1)
].sort_values('lift')

print(f'Reglas posibles:                        {len(reglas):,}')
print(f'Reglas útiles (asociación positiva):    {len(reglas_utiles):,}')
print(f'Parejas que se compran juntas MENOS de lo esperado (lift < 1): {len(reglas_negativas)}')
```

```
Reglas posibles:                        1,560
Reglas útiles (asociación positiva):    258
Parejas que se compran juntas MENOS de lo esperado (lift < 1): 100
```

🔧 Presentación de la tabla

- Todas las proporciones se muestran en **porcentaje** y con el sufijo `_%` en el nombre, para que no haya dudas. Antes `soporte_a` y `confianza` estaban en porcentaje pero el lift no, y no se indicaba.
- El **lift se redondea a 2 decimales** (antes a 1, y así 0,96 y 1,04 salían ambos como 1,0).
- 🔧 El comentario original decía "ordenar por confianza", pero el código ordenaba por lift. Ahora se ordena por **lift** (fuerza de la asociación) y, a igualdad de lift, por confianza.

```
def formatear(tabla):
    salida = tabla[['antecedente', 'consecuente', 'pedidos_ac']].copy()
    for col in ['soporte_a', 'soporte_c', 'soporte_ac', 'confianza']:
        salida[f'{col}_%'] = (tabla[col] * 100).round(2)
    salida['lift'] = tabla['lift'].round(2)
    return salida

tabla_reglas = formatear(reglas_utiles)
tabla_reglas.head(20)
```

	antecedente	consecuente	pedidos_ac	soporte_a_%	soporte_c_%	soporte_ac_%	confianza_%	lift
0	Cilantro Orgánico	Limones	18527	3.54	7.11	0.90	25.41	3.58
1	Limones	Cilantro Orgánico	18527	7.11	3.54	0.90	12.65	3.58
2	Ajo Orgánico	Cebolla Amarilla Orgánica	22886	5.52	5.71	1.11	20.12	3.53
3	Cebolla Amarilla Orgánica	Ajo Orgánico	22886	5.71	5.52	1.11	19.47	3.53
4	Apio Orgánico en Ramillete Pequeño	Cebolla Amarilla Orgánica	11456	3.43	5.71	0.56	16.22	2.84
5	Cebolla Amarilla Orgánica	Apio Orgánico en Ramillete Pequeño	11456	5.71	3.43	0.56	9.74	2.84
6	Cebolla Roja Orgánica	Ajo Orgánico	10514	3.43	5.52	0.51	14.88	2.69
7	Ajo Orgánico	Cebolla Roja Orgánica	10514	5.52	3.43	0.51	9.24	2.69
8	Limón Grande	Limones	28941	7.79	7.11	1.40	18.03	2.54
9	Limones	Limón Grande	28941	7.11	7.79	1.40	19.76	2.54
10	Zanahorias	Cebolla Amarilla Orgánica	10477	3.65	5.71	0.51	13.94	2.44
11	Cebolla Amarilla Orgánica	Zanahorias	10477	5.71	3.65	0.51	8.91	2.44
12	Cebolla Roja Orgánica	Limones	11448	3.43	7.11	0.56	16.20	2.28
13	Limones	Cebolla Roja Orgánica	11448	7.11	3.43	0.56	7.82	2.28
14	Limón Orgánico	Aguacate Hass Orgánico	21997	4.42	10.70	1.07	24.14	2.26
15	Aguacate Hass Orgánico	Limón Orgánico	21997	10.70	4.42	1.07	9.98	2.26
16	Limón Orgánico	Ajo Orgánico	11048	4.42	5.52	0.54	12.13	2.20
17	Ajo Orgánico	Limón Orgánico	11048	5.52	4.42	0.54	9.71	2.20
18	Limón Orgánico	Cebolla Amarilla Orgánica	10989	4.42	5.71	0.53	12.06	2.11
19	Cebolla Amarilla Orgánica	Limón Orgánico	10989	5.71	4.42	0.53	9.35	2.11

Productos que se compran juntos *menos* de lo esperado (lift < 1)

No sirven para recomendar, pero dan información útil: suelen ser **productos sustitutivos** (el cliente elige uno u otro).

```
# Cada pareja aparece dos veces (A → B y B → A) con el mismo lift y los mismos pedidos:
# para esta tabla nos quedamos con una de las dos
formatear(reglas_negativas).drop_duplicates(subset=['lift', 'pedidos_ac']).head(10)
```

	antecedente	consecuente	pedidos_ac	soporte_a_%	soporte_c_%	soporte_ac_%	confianza_%	lif
259	Banana	Limón Orgánico	11231	23.81	4.42	0.55	2.29	0.5
44	Aguacate Hass Orgánico	Banana	32126	10.70	23.81	1.56	14.57	0.6
357	Cebolla Amarilla Orgánica	Banana	17343	5.71	23.81	0.84	14.75	0.6
787	Fresas	Bolsa de Bananas Orgánicas	18347	7.24	19.13	0.89	12.30	0.6
161	Apio Orgánico en Ramillete Pequeño	Banana	11021	3.43	23.81	0.53	15.61	0.6
266	Banana	Pepino Orgánico	13311	23.81	4.11	0.65	2.71	0.6
297	Bolsa de Bananas Orgánicas	Limón Grande	20424	19.13	7.79	0.99	5.18	0.6
245	Banana	Cilantro Orgánico	11599	23.81	3.54	0.56	2.36	0.6
300	Bolsa de Bananas Orgánicas	Manzana Fuji Orgánica	12046	19.13	4.50	0.58	3.06	0.6
84	Aguacate Orgánico	Bolsa de Bananas Orgánicas	23910	8.93	19.13	1.16	13.00	0.6

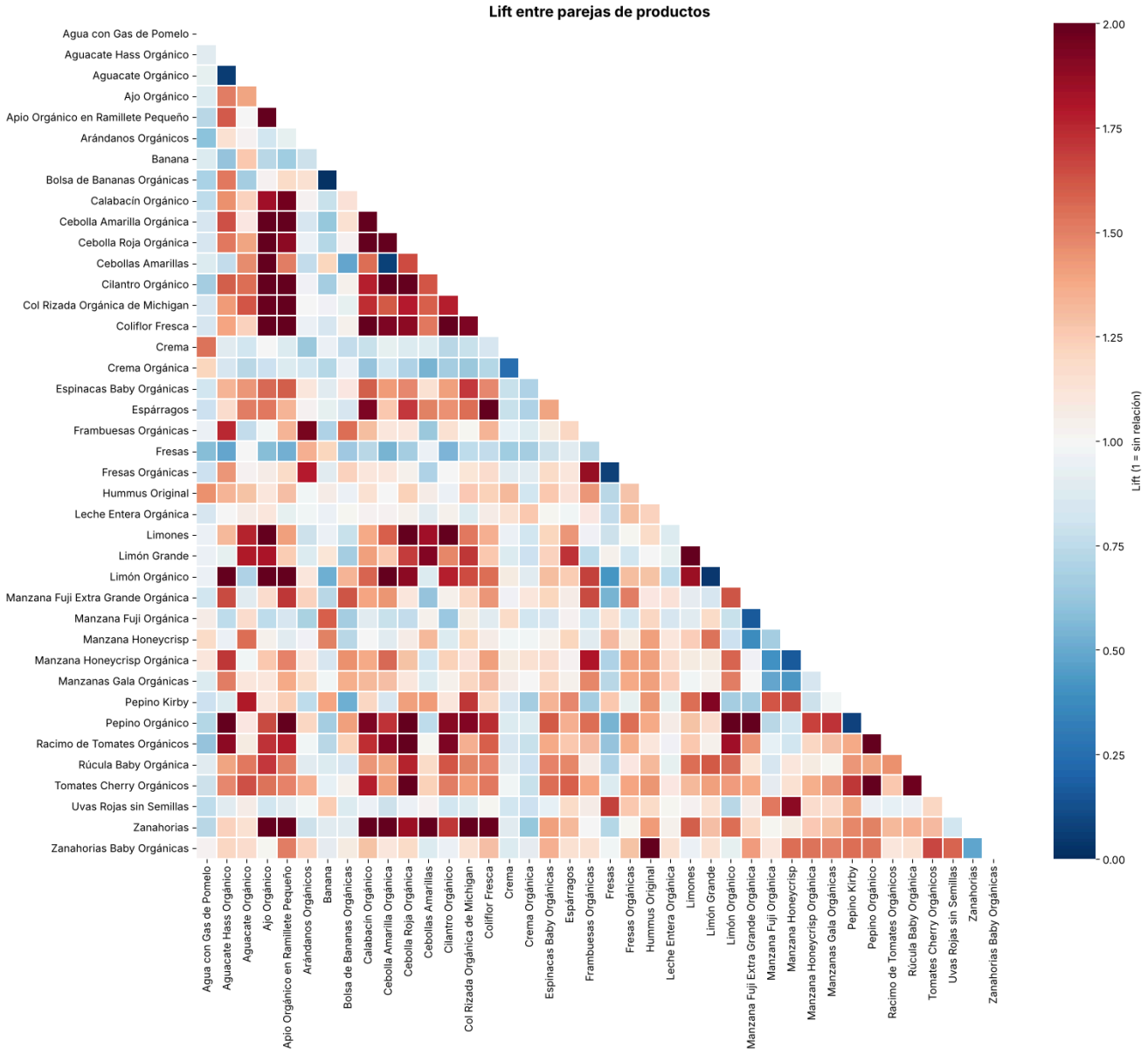
9. Visualización

Mapa de calor del lift entre productos

Cada casilla es el lift entre dos productos (es simétrico, así que se muestra solo la mitad). **Rojo**: se compran juntos más de lo esperado. **Azul**: menos de lo esperado. Blanco: sin relación.

```
matriz_lift = (reglas.pivot(index='antecedente', columns='consecuente', values='lift')
               .reindex(index=productos, columns=productos))
mascara = np.triu(np.ones_like(matriz_lift, dtype=bool))   # oculta la mitad repetida y la di

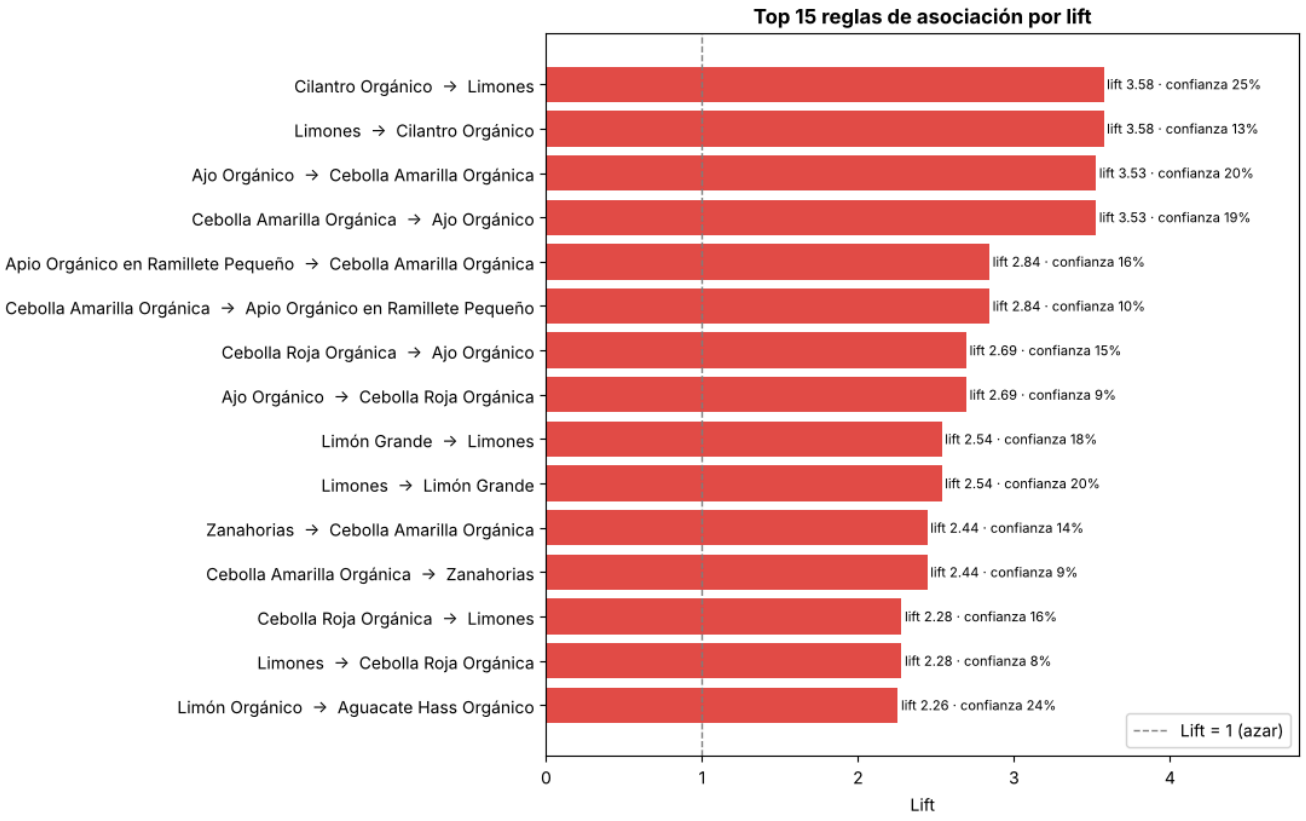
fig, ax = plt.subplots(figsize=(16, 14))
sns.heatmap(matriz_lift, mask=mascara, cmap='RdBu_r', center=1, vmin=0, vmax=2,
            linewidths=0.3, cbar_kws={'label': 'Lift (1 = sin relación)'}, ax=ax)
ax.set_title('Lift entre parejas de productos', fontsize=14, fontweight='bold')
ax.set_xlabel('')
ax.set_ylabel('')
plt.tight_layout()
plt.show()
```



Las 15 reglas con mayor lift

```
top15 = reglas_utiles.head(15).iloc[::-1]
etiquetas = top15['antecedente'] + '  →  ' + top15['consecuente']

fig, ax = plt.subplots(figsize=(11, 7))
barras = ax.barh(etiquetas, top15['lift'], color='#E24B4A')
ax.axvline(1, color='gray', ls='--', lw=1, label='Lift = 1 (azar)')
for barra, conf in zip(barras, top15['confianza']):
    ax.text(barra.get_width() + 0.02, barra.get_y() + barra.get_height() / 2,
            f'lift {barra.get_width():.2f} · confianza {conf:.0%}', va='center', fontsize=8)
ax.set_xlim(0, top15['lift'].max() * 1.35)
ax.set_xlabel('Lift')
ax.set_title('Top 15 reglas de asociación por lift', fontweight='bold')
ax.legend(loc='lower right')
plt.tight_layout()
plt.show()
```



10. Enriquecer las reglas con los datos de producto

Se añaden `id_producto`, `id_seccion` e `id_departamento` para poder cruzar las reglas con otras tablas (por ejemplo, en Power BI).

🔧 Correcciones:

1. **Antes solo se enriquecía el antecedente.** Ahora se añaden los datos del antecedente **y** del consecuente (sufijos `_a` y `_c`).
2. **Antes se renombraban las columnas por posición** (`df.columns = [...]`). Si el orden de las columnas cambiaba, los nombres quedaban mal puestos sin dar ningún error. Ahora se renombran por nombre.
3. `validate='many_to_one'` en el `merge`: si un mismo nombre de producto tuviera dos `id_producto`, cada regla se duplicaría en silencio. Con `validate`, pandas da un error en vez de duplicar filas.

```
productos_unicos = (df[['nombre_producto', 'id_producto', 'id_seccion', 'id_departamento']]
                    .drop_duplicates()
                    .sort_values('nombre_producto')
                    .reset_index(drop=True))
print(f'Productos únicos: {len(productos_unicos)}')
productos_unicos
```

```
Productos únicos: 40
```

	nombre_producto	id_producto	id_seccion	id_departamento
0	Agua con Gas de Pomelo	44632	115	7
1	Aguacate Hass Orgánico	47209	24	4
2	Aguacate Orgánico	47766	24	4
3	Ajo Orgánico	24964	83	4
4	Apio Orgánico en Ramillete Pequeño	44359	83	4
5	Arándanos Orgánicos	39275	123	4
6	Banana	24852	24	4
7	Bolsa de Bananas Orgánicas	13176	24	4
8	Calabacín Orgánico	45007	83	4
9	Cebolla Amarilla Orgánica	22935	83	4
10	Cebolla Roja Orgánica	8518	83	4
11	Cebollas Amarillas	4605	83	4
12	Cilantro Orgánico	31717	16	4
13	Col Rizada Orgánica de Michigan	28985	83	4
14	Coliflor Fresca	27104	83	4
15	Crema	27086	53	16
16	Crema Orgánica	49235	53	16
17	Espinacas Baby Orgánicas	21903	123	4
18	Espárragos	46979	83	4
19	Frambuesas Orgánicas	27966	123	4
20	Fresas	16797	24	4
21	Fresas Orgánicas	21137	24	4
22	Hummus Original	30489	67	20
23	Leche Entera Orgánica	27845	84	16
24	Limones	26209	24	4
25	Limón Grande	47626	24	4
26	Limón Orgánico	5876	24	4
27	Manzana Fuji Extra Grande Orgánica	19057	24	4
28	Manzana Fuji Orgánica	28204	24	4
29	Manzana Honeycrisp	45066	24	4
30	Manzana Honeycrisp Orgánica	8277	24	4
31	Manzanas Gala Orgánicas	37646	24	4
32	Pepino Kirby	49683	83	4
33	Pepino Orgánico	30391	83	4
34	Racimo de Tomates Orgánicos	41950	83	4
35	Rúcula Baby Orgánica	21616	123	4
36	Tomates Cherry Orgánicos	40706	123	4
37	Uvas Rojas sin Semillas	4920	123	4

	nombre_producto	id_producto	id_seccion	id_departamento
38	Zanahorias	17794	83	4
39	Zanahorias Baby Orgánicas	42265	123	4

```
def datos_producto(sufijo):
    return productos_unicos.rename(columns={
        'nombre_producto': 'antecedente' if sufijo == 'a' else 'consecuente',
        'id_producto': f'id_producto_{sufijo}',
        'id_seccion': f'id_seccion_{sufijo}',
        'id_departamento': f'id_departamento_{sufijo}',
    })

reglas_enriquecidas = (tabla_reglas
                       .merge(datos_producto('a'), on='antecedente', how='left', validate='ma
                       .merge(datos_producto('c'), on='consecuente', how='left', validate='ma

assert len(reglas_enriquecidas) == len(tabla_reglas), 'El merge ha duplicado filas'
assert reglas_enriquecidas[['id_producto_a', 'id_producto_c']].notna().all().all(), 'Hay proc
print('✅ Reglas enriquecidas sin duplicados ni huecos')
reglas_enriquecidas.head(10)
```

```
✅ Reglas enriquecidas sin duplicados ni huecos
```

	antecedente	consecuente	pedidos_ac	soporte_a_%	soporte_c_%	soporte_ac_%	confianza_%	lift
0	Cilantro Orgánico	Limones	18527	3.54	7.11	0.90	25.41	3.58
1	Limones	Cilantro Orgánico	18527	7.11	3.54	0.90	12.65	3.58
2	Ajo Orgánico	Cebolla Amarilla Orgánica	22886	5.52	5.71	1.11	20.12	3.53
3	Cebolla Amarilla Orgánica	Ajo Orgánico	22886	5.71	5.52	1.11	19.47	3.53
4	Apio Orgánico en Ramillete Pequeño	Cebolla Amarilla Orgánica	11456	3.43	5.71	0.56	16.22	2.84
5	Cebolla Amarilla Orgánica	Apio Orgánico en Ramillete Pequeño	11456	5.71	3.43	0.56	9.74	2.84
6	Cebolla Roja Orgánica	Ajo Orgánico	10514	3.43	5.52	0.51	14.88	2.69
7	Ajo Orgánico	Cebolla Roja Orgánica	10514	5.52	3.43	0.51	9.24	2.69
8	Limón Grande	Limones	28941	7.79	7.11	1.40	18.03	2.54
9	Limones	Limón Grande	28941	7.11	7.79	1.40	19.76	2.54

Nombre de la sección (archivo `secciones.xlsx`)

El archivo `secciones.xlsx` relaciona cada `id_seccion` con su **nombre** (Frutas, Verduras, Lácteos...) y la **URL de una imagen** (útil para mostrarla en Power BI).

Con el nombre de la sección se pueden hacer preguntas como *"¿qué secciones se compran juntas?"* sin tener que recordar qué es la sección 83.

- Si el archivo no está en la carpeta, este paso se salta y el resto del notebook funciona igual.
- Se comprueba que **todas** las secciones de las reglas tienen nombre.
- ⚠️ En el Excel, la sección 123 se llama "Organic", en inglés, mientras que las demás están en español. Si te molesta en los informes, corrígelo en el Excel.

```
if RUTA_SECCIONES.exists():
    secciones = pd.read_excel(RUTA_SECCIONES)
    print(secciones.to_string(index=False))

    for sufijo in ['a', 'c']:
        reglas_enriquecidas = reglas_enriquecidas.merge(
            secciones.rename(columns={'id_seccion': f'id_seccion_{sufijo}',
                                      'nombre_seccion': f'nombre_seccion_{sufijo}',
                                      'imagen': f'imagen_seccion_{sufijo}'}),
            on=f'id_seccion_{sufijo}', how='left', validate='many_to_one')

    sin_nombre = reglas_enriquecidas[['nombre_seccion_a', 'nombre_seccion_c']].isna().any(axi
    assert sin_nombre == 0, f'{sin_nombre} reglas tienen una sección que no está en {RUTA_SEC
    print('\n✅ Todas las reglas tienen el nombre de sección del antecedente y del consecuent
else:
    print(f'No se encuentra {RUTA_SECCIONES}: las reglas se exportan sin el nombre de la secc
```

```
 id_seccion nombre_seccion                          imagen
         16        Hierbas https://i.imgur.com/WhPWzCi.jpeg
         24         Frutas https://i.imgur.com/pGfi3gl.jpeg
         53         Cremas https://i.imgur.com/OzlSaxb.jpeg
         67       Untables https://i.imgur.com/cJcbz4R.jpeg
         83       Verduras https://i.imgur.com/OvWTLNC.jpeg
         84        Lácteos https://i.imgur.com/yF9qMjB.jpeg
        115        Bebidas https://i.imgur.com/Qu7Whn3.jpeg
        123        Organic https://i.imgur.com/4ZrkyYM.jpeg

✅ Todas las reglas tienen el nombre de sección del antecedente y del consecuente
```

```
# ¿Qué secciones se compran juntas? Número de reglas útiles entre cada par de secciones
if 'nombre_seccion_a' in reglas_enriquecidas:
    display(pd.crosstab(reglas_enriquecidas['nombre_seccion_a'], reglas_enriquecidas['nombre_
                        rownames=['Sección antecedente'], colnames=['Sección consecuente']))
```

Sección consecuente	Frutas	Hierbas	Lácteos	Organic	Verduras
Sección antecedente					
Frutas	59	2	3	28	32
Hierbas	3	0	0	1	0
Lácteos	3	0	0	2	0
Organic	28	0	2	12	10
Untables	3	0	0	1	0
Verduras	44	0	0	13	12

11. Exportación a CSV

Se mantiene el formato del original para Power BI/Excel en español (`sep=';'` y `decimal=','`).

🔧 Se añade **`encoding='utf-8-sig'`**. Sin él, Excel abre el archivo con otra codificación y los nombres con tilde salen mal ("OrgÃ¡nica" en vez de "Orgánica").

Al final se vuelve a leer el archivo para **comprobar** que se ha guardado bien.

```
reglas_enriquecidas.to_csv(RUTA_CSV_REGLAS, index=False, sep=';', decimal=',', encoding='utf-

comprobacion = pd.read_csv(RUTA_CSV_REGLAS, sep=';', decimal=',', encoding='utf-8-sig')
assert len(comprobacion) == len(reglas_enriquecidas)
assert comprobacion['lift'].dtype == float
print(f'✅ {RUTA_CSV_REGLAS} guardado: {len(comprobacion):,} reglas, {comprobacion.shape[1]}
print('Ejemplo de nombre con tilde leído de nuevo:',
      next((n for n in comprobacion['antecedente'] if 'á' in n or 'ú' in n), '(ninguno)'))
```

```
✅ reglas_market_basket.csv guardado: 258 reglas, 18 columnas
Ejemplo de nombre con tilde leído de nuevo: Cilantro Orgánico
```

12. Cómo interpretar los resultados

Ejemplo con los datos reales

La regla más fuerte es **Cilantro Orgánico → Limones**, con confianza 25,41 % y lift 3,58:

- **Confianza 25,41 %**: de cada 4 pedidos con cilantro, en 1 hay también limones.
- **Lift 3,58**: un cliente que compra cilantro tiene **3,58 veces más probabilidad** de comprar limones que un cliente cualquiera. Es una asociación fuerte (ingredientes de una misma receta).

Ahora también está la regla inversa, **Limones → Cilantro Orgánico**, con el **mismo lift** pero **otra confianza** (12,65 %), porque los limones se compran más que el cilantro. Con `combinations` solo se habría calculado una de las dos.

Cómo usar cada métrica

Si quieres...	Mira	Por qué
Saber si una asociación es real y no casualidad	**Lift > 1**	Compara con lo esperable por azar
Recomendar B a quien compra A	**Confianza (A → B)**	Es la probabilidad de acertar con la recomendación
Saber si merece la pena actuar	**Soporte de la regla**	Una regla fuerte pero muy rara afecta a pocos pedidos

Acciones de negocio típicas

- **Lift alto + soporte alto** → colocar juntos en tienda y web, packs o recetas.
- **Lift alto + confianza alta** → recomendaciones automáticas ("quien compra A, también compra B").
- **Lift < 1** → productos sustitutivos: no promocionarlos juntos.

Limitaciones

- Las reglas indican que dos productos **se compran juntos**, no que uno **cause** la compra del otro.
- Solo se analizan **parejas** (A → B). Para reglas con varios productos ({A, B} → C) se usa el algoritmo **Apriori** (librería `mlxtend`), que sigue la misma lógica de soporte, confianza y lift.

Resumen de correcciones

Problema en la versión anterior	Solución
`combinations` → solo un sentido de cada regla (faltaba la mitad)	Todas las parejas ordenadas (A → B y B → A)
Reglas con lift < 1 mezcladas con las útiles	Filtro `lift > 1` y tabla aparte de productos sustitutivos
Sin soporte de la regla	Columnas `pedidos_ac` y `soporte_ac_%`, y umbral mínimo
Lift redondeado a 1 decimal	2 decimales
Unidades mezcladas sin indicar	Sufijo `_%` en las columnas en porcentaje
Comentario "ordenar por confianza" pero ordenaba por lift	Orden por lift y confianza, y comentario coherente
Unir con comas y `str.get_dummies(sep=',')`	Matriz con `factorize`, sin depender de comas y 8 veces menos memoria
Cálculo muy lento (filtrar millones de filas por pareja)	Una multiplicación de matrices `X.T @ X`
Sin comprobación de que el nuevo cálculo es correcto	Comparación con las funciones originales
`sqlite3.connect` crea una BD vacía si falta el archivo	Comprobación previa del archivo
Conexión sin cerrar si la consulta falla	`with closing(...)`
`SELECT *`	Solo las columnas necesarias
Sin revisión de calidad de datos	Nulos, coherencia de precios, repetidos e ids
Solo se enriquecía el antecedente	Antecedente y consecuente
Sin nombre de sección	Nombre e imagen de la sección desde `secciones.xlsx`
Columnas renombradas por posición	Renombrado por nombre y `validate` en el `merge`
CSV sin `utf-8-sig` (tildes mal en Excel)	`encoding='utf-8-sig'` y lectura de comprobación
Erratas: "subonjunto", "prodcutos", "centa"	Corregidas